

Premo: Integrated Versioning

Stuart Bryson

DreamWorks

Australia

stuart.bryson@dreamworks.com

Matthew C. Gong

DreamWorks

United States of America

matt.gong@dreamworks.com

Ben Kwa

DreamWorks

United States of America

ben.kwa@dreamworks.com

CCS CONCEPTS

• Computing methodologies → Animation.

KEYWORDS

rigging, data model, authoring, animation

ACM Reference Format:

Stuart Bryson, Matthew C. Gong, and Ben Kwa. 2024. Premo: Integrated Versioning. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH Talks '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641233.3664329>

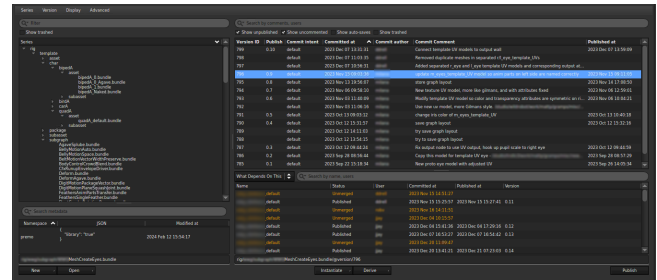
1 INTRODUCTION

Premo is a SciTech Award winning animation application developed by DreamWorks. Recently, rigging was incorporated into the application. As introduced in SIGGRAPH 2023 [Bryson et al. 2023], Premo supports a fully integrated versioning experience. Integrated versioning allows Premo to provide native workflows for version locking and automatic updates, rolling back, and diff/merge. By integrating versioning into the application, Premo is able to empower its artists by working at a higher dimension than simply the files on disk. Premo can better reason about the different versions of data and provide enhanced workflows to better support its artists. This talk takes a deeper dive into understanding the motivations of integrated versioning and how it was implemented in Premo.

2 DATA REFERENCING

Premo's data model makes heavy use of referencing. A shot may reference and instantiate multiple characters. Each character may reference and derive from a common template such as a Biped. The Biped may in turn reference and instantiate other data such as limbs, hands, fingers, face feature lines, and more. This data referencing approach allows for powerful and comprehensive reuse of character components across multiple productions. This drastically reduces the amount of work required to create new characters and maintain existing ones.

Each version of a definition is given a unique ID by the underlying versioning system. When one definition references another definition, it does so by pointing at a specific version of that definition. This ensures the integrity of the data even when newer



Name	Version	Created at	Published at
Biped	1.0.0	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.1	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.2	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.3	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.4	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.5	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.6	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.7	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.8	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.9	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.10	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.11	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.12	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.13	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.14	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.15	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.16	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.17	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.18	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.19	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.20	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.21	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.22	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.23	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.24	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.25	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.26	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.27	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.28	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.29	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.30	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.31	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.32	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.33	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.34	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.35	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.36	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.37	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.38	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.39	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.40	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.41	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.42	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.43	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.44	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.45	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.46	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.47	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.48	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.49	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.50	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.51	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.52	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.53	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.54	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.55	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.56	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.57	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.58	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.59	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.60	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.61	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.62	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.63	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.64	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.65	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.66	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.67	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.68	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.69	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.70	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.71	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.72	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.73	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.74	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.75	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.76	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.77	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.78	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.79	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.80	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.81	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.82	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.83	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.84	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.85	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.86	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.87	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.88	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.89	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.90	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.91	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.92	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.93	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.94	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.95	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.96	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.97	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.98	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.99	2023-07-11 10:10:10	2023-07-11 10:10:10
Biped	1.0.100	2023-07-11 10:10:10	2023-07-11 10:10:10

Figure 1: Property of DreamWorks

versions become available. Along with the unique version ID, versions can also be given a publish number that is unique within each series. Each publish number contains a major and a minor field (e.g. 4.5). Artists typically use minor numbers as non-breaking changes, while major numbers are used to communicate broad changes to the definition and are often not backward compatible with the previous version.

Managing all the various mutations and permutations of such a reference graph can be challenging and especially so if there is no data versioning system to assist. Most digital content creation (DCC) software have little, or no support for data versioning. Some may allow you to reference and instantiate certain external data, but that data needs to be reified onto disk before loading the referencing scene; and further, these DCCs usually don't support instancing multiple versions of the same data. Premo's integrated versioning allows for these features and more.

3 VERSIONING WORKFLOWS

By integrating versioning into the application, the artist gains many new workflows previously unavailable in rigging and animation DCCs.

3.1 Automatic Update and Version Locking

One such workflow is the ability to automatically update to the latest version of referenced data. For example, an animator may want to always use the latest version of a character in their shot. As soon as the character rigger creates a new version of the character, the animator will automatically receive that update either in session, or the next time they load. Or a character rigger working on the Biped may want to automatically receive an update to the Arm definition. Automatic update is a recursive feature that enables the Biped to update to the latest Hand definition, even when the Hand is only indirectly referenced by the Biped.

Automatic update works in conjunction with a flexible set of locking rules. These version locking rules are, amongst other criteria, based on the publish number. Artists can choose to lock a

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH Talks '24, July 27–August 01, 2024, Denver, CO, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0515-1/24/07

<https://doi.org/10.1145/3641233.3664329>

reference to a particular minor version (e.g. 4.5), or relax the lock a little by locking to just the major version (e.g. 4.*), or leave the reference unlocked entirely. When the auto-update process runs, the minor locked references will not change which definition they are looking at, whilst the major lock will update any reference to the latest minor version of the existing major version, and finally the unlocked version will update to the latest major and minor version.

Typically artists will leave references entirely unlocked during early development of the character rigs and shots, switch to major locks as they start real shot work to avoid receiving disruptive changes, and lock on minor versions as the shots or show near completion.

At any point that a new version contains bugs, incompatible features, or unintended behavior, the artist can easily roll back the reference to a known working version and continue working. Artists can also cherry-pick a subset of data from previous versions. Animators use this feature extensively when reverting to previous animation. Further, the cherry-picking doesn't have to be from the same series - the artists can extract data from entirely unrelated series, for example a rigger may copy a cage model from one character to another.

3.2 Multiple Versions

Unlike many external revision control systems, Premo supports referencing different versions of the same series at the same time. In the previous example, where an animator locks one character but not the others, the locked character may be referencing an older version of the Finger definition while the unlocked characters may be referencing a newer version. This can be useful when specific characters need to retain the existing behavior while others may benefit from the improvements found in new versions of referenced definitions.

3.3 Diff and Merge

Premo provides the ability to diff and merge multiple definitions, including identifying the ancestor version for a three-way merge. It's important to emphasize that the diff and merge occurs in a higher dimension than just the definition files on disk. Rather, Premo diffs at the Overrides data model level - where each side of the diff contains the resolved state of multiple layers of definitions. Diffing individual definition files would be untenable and another reason why integrated versioning is so critical.

4 IMPLEMENTATION

Premo serializes the bulk of the data to a file system while the relationship between the data, as well as metadata, is stored in a database. By serializing the data to the file system, artists and pipeline engineers can easily inspect the data by browsing files on disk. And by storing the relationships between the data in a database, Premo can perform fast queries to reason about the data. For example, it can find the full closure of series and versions required to open a definition.

Further, Premo supports multiple types of version repositories. For large productions, Premo uses a centralized Postgres database

and shared file system. Premo can be pointed at a Postgres repository so that multiple sessions can consume and contribute to the same data set. These sessions are also notified when new versions are created in the repository, or when metadata is modified. For small productions or non-production workflows, such as developer application testing, rigger prototyping, unit or integration testing, Premo can use an SQLite repository. The advantage of using an SQLite repository is it requires no setup whereas the Postgres setup requires a Postgres instance to be set up. The repositories are plug-gable so connecting to a different database type would be simply a matter of writing the connector for that database type.

5 DATA TRANSFER

Premo can also transfer data between repositories. This is useful when archiving data, or sharing data between multiple productions or studios. Data transfers can be targeted to a subset of the repository and ensures all referenced data is transferred to ensure data integrity is maintained. The transfer can be between different repository types, for example transferring data from a Postgres repository to a local SQLite repository for isolated testing.

6 DATA INTEGRITY

Finally, the Premo versioning system is complemented by a separate process that enforces data integrity between the file system and the database. This process wraps the repository file system and adjusts file ownership and permissions to protect data from being accidentally modified or deleted. It is implemented as a simple wrapper around the FUSE virtual file system.

7 RESULTS

Versioning in Premo was first deployed for the animated holiday special *Trolls: Holiday in Harmony* and then on *Trolls: Band Together*. Versioning quickly became an integral part of the artists process, unlocking workflows that were previously not possible. Rigging artists were able to iterate on character modules quickly, making improvements and bug fixes efficiently, tracking their changes across versions, and controlling the deployment of changes using the flexible automatic update and version locking rules.

With Premo's powerful and integrated versioning workflows, along with its scalable and flexible implementation, we believe Premo has the best solution to production data versioning in the industry. As a result our artists are more productive and efficient using Premo than any other tool available.

ACKNOWLEDGMENTS

This effort was supported by the artists, developers, and executives at DreamWorks with a special recognition of the Production Software AnimRig and the Rigging CharTech teams.

REFERENCES

- Stuart Bryson, Matthew Gong, Rob O'Neill, Ted Forgrave, Ben Kwa, and Paul DiLorenzo. 2023. Premo: Powerful Character Rigging, Fast Animation. In *SIGGRAPH '23: ACM SIGGRAPH 2023 Talks*. 1–2. <https://doi.org/10.1145/3587421.3595468>